

Tutorial 13

Exercise 1: Subset sum

Give a dynamic programming algorithm for Subset Sum. To this end, specify the recurrence relation (including initial values).

Solution:

Given an instance $n_1, \dots, n_k \in \mathbb{N}$ and $t \in \mathbb{N}$, define

$$F(j, t') \begin{cases} 1 & \text{there is a subset of } n_1, \dots, n_j \text{ that sums up to } t' \\ 0 & \text{otherwise.} \end{cases}$$

for $0 \leq j \leq k$ and $0 \leq t' \leq t$.

This is captured by the recurrence relation

$$F(j, t') = \begin{cases} F(j-1, t' - n_j) \vee F(j-1, t') & \text{if } t' \geq n_j \\ F(j-1, t') & \text{otherwise} \end{cases}$$

with

$$F(j, 0) = 1 \text{ for all } j \geq 0$$

and

$$F(0, t) = 0 \text{ for all } t > 0$$

stating that one can sum up to t' with a subset of $\{n_1, \dots, n_j\}$ by either

- taking n_j into the sum and summing up to $t' - n_j$ with a subset of $\{n_1, \dots, n_{j-1}\}$, or
- by not taking n_j into the sum and summing up to t' with a subset of $\{n_1, \dots, n_{j-1}\}$.

The algorithm computes all values $F(j, t')$ and returns $F(k, t)$.

Exercise 2: Matrix multiplication

Multiplying an $n \times m$ -matrix with an $m \times o$ -matrix takes nmo multiplications of numbers using the naive matrix multiplication algorithm. Furthermore, matrix multiplication is associative, i.e., $A(BC) = (AB)C$ for matrices A, B, C of the right sizes. While the results of the different ways of multiplying the matrices are the same, the number of multiplications is not necessarily the same. For example, if A is a 10×30 -matrix, B is a 30×5 -matrix, and C is a 5×60 -matrix, then

- computing $(AB)C$ takes 4500 multiplications while
- computing $A(BC)$ takes 27.000 multiplications.

The matrix multiplication problem is, given a sequence of n matrices $A_1, A_2, \dots, A_n$ that can be multiplied as $A_1 A_2 \dots A_n$, to determine the minimal number of number multiplications necessary to compute the product (but not to compute the product).

Give a dynamic programming algorithm for the matrix multiplication problem. To this end, specify the recurrence relation (including initial values).

Solution:

Given an instance $A_1, \dots, A_n$ matrices, let $m_0, m_1, \dots, m_n$ such that A_j is an $m_{j-1} \times m_j$ -matrix (such m_j exist, as we assume that the matrices can be multiplied).

Now, define $F(i, j)$ for $1 \leq i \leq j \leq n$ to be the minimal number of multiplications necessary to compute $A_i \dots A_j$. Then, we have $F(i, i) = 0$ for all i and

$$F(i, j) = \min\{F(i, k) + F(k+1, j) + m_{k-1}m_k m_{k+1} \mid i \leq k < j\}$$

for $i < j$ signifying that to compute $A_i \cdots A_j$ for $i < k$ we need to first compute $A_i \cdots A_k$ (with minimal number of multiplications $F(i, k)$) and $A_{k+1} \cdots A_j$ (with minimal number of multiplications $F(k+1, j)$) and then multiply the results, which takes $m_{i-1}m_k m_j$ multiplications.

The algorithm computes all values $F(i, j)$ and returns $F(1, n)$.

Exercise 3: Challenge

Solve Day 16 of the Advent of Code 2022: <https://adventofcode.com/2022/day/16>. Use the input given on Moodle.

Solution:

The dynamic programming idea is to compute $F(v, t, O)$, which is defined as the maximal amount of pressure that can be released when starting at valve v with time $t \leq 30$ left and the valves in O already open. Note that many valves have flow rate 0, so they are useless and can be removed! Thus, O cannot get too large.

$F(v, t, O)$ can be expressed by a recurrence relation taking into account the time it takes to move to a valve and the time it takes to open the valve (note that not every valve has a direct tunnel to every other valve). So, we compute all these values and return $F(v_0, 30, \emptyset)$ where v_0 is the initial valve.